COURSE SLIDES · WSQ

Develop Multi-Agent AI Applications with AutoGen

WSQ Course Code: TGS-2024045806

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v9 · 28 August 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

Digital Attendance (Mandatory)

1

Three times a day

Take the AM, PM and Assessment digital attendance — mandatory for every WSQ-funded course.

2

Trainer shows the QR

The trainer or administrator displays the digital attendance QR code from the SSG portal.

3

Scan and submit

Scan the QR code with your mobile phone camera and submit your attendance.

4

75% minimum

A minimum of 75% attendance is required to be eligible for assessment and funding.

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Dr. Alfred Ang

Principal Trainer
Tertiary Infotech Academy Pte Ltd

ROLE
Principal Trainer and Founder, Tertiary Infotech Academy Pte Ltd

EXPERTISE
Generative AI, Large Language Models, multi-agent AI systems and applied machine learning.

DELIVERS
WSQ courses on AI agents, LLM application development, RAG, data science and automation.

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

- Your name, organisation and role.
- Your experience with Python, APIs and Generative AI tools (if any).
- One workflow you would like a team of AI agents to automate for you.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. 75% attendance is required.

© 2026 Tertiary Infotech Academy Pte Ltd

LMS / TMS · lms-tms.tertiaryinfotech.com

Download Your Course Material

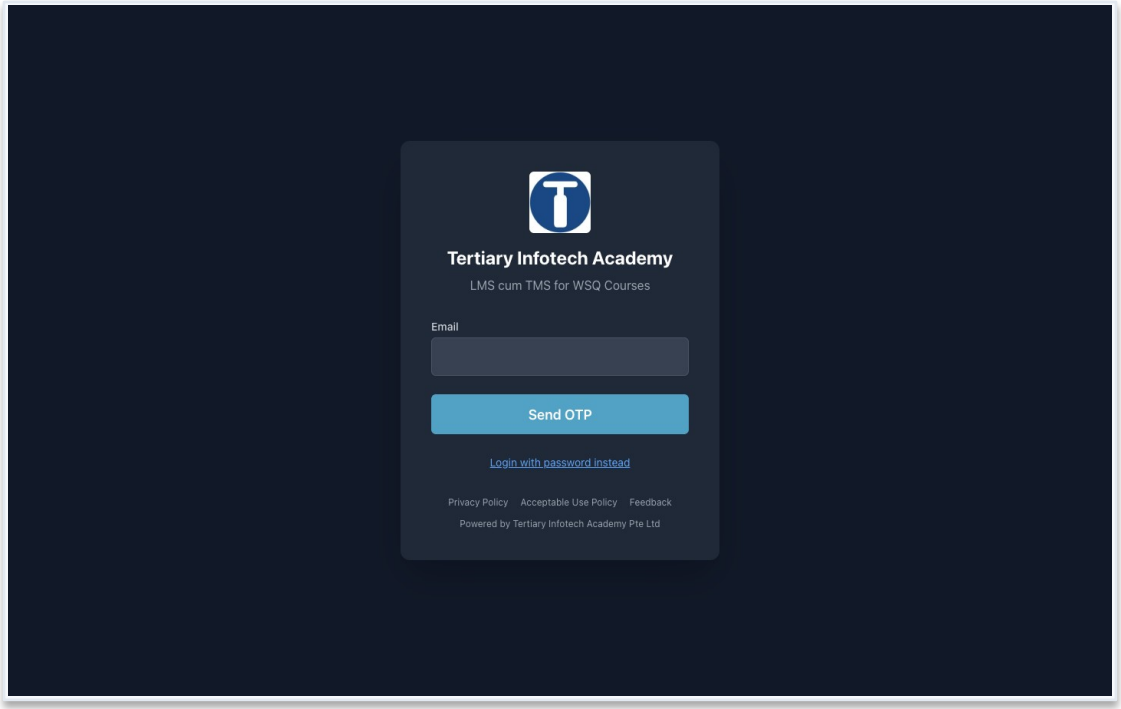
1. **Go to the LMS portal**
   Open https://lms-tms.tertiaryinfotech.com in your browser.

2. **Log in**
   Sign in with the e-mail you used to register (OTP or password).

3. **Open this course**
   Select 'Develop Multi-Agent AI Applications with AutoGen'.

4. **Download the materials**
   Trainer Slides, Learner Guide and Lesson Plan (PDF).

5. **Get the labs**
   Clone the GitHub lab repository — the link is on the course page.



© 2026 Tertiary Infotech Academy Pte Ltd

Skills Framework Alignment

1

TSC Title

Artificial Intelligence Application

2

TSC Code

AER-TEM-4026-1.1

3

A1 / A2

Analyse algorithms in AI applications; correlate algorithm design with efficiency.

4

A3 / A4

Identify strengths and limitations of AI applications; evaluate various AI applications.

5

A5 / A6

Assess the feasibility of AI applications and the improvements they bring to processes.

6

K1–K6

AI application ranges, performance analysis, algorithm design and implementation concepts.

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 2 Days, 8 Hours per Day

Day 1

- Day 1 — LLM Orchestration & RAG with AutoGen

– Digital Attendance (AM) · Introductions · Learning Outcomes

– Topic 1: Introduction to LLM AI Orchestration (Labs 1–3)

– Lunch Break · Digital Attendance (PM)

– Topic 2: Retrieval-Augmented Generation (Labs 4–5)

– Day 1 recap and Q&A

Day 2

- Day 2 — Multi-Agent Workflows & Assessment

– Digital Attendance (AM) · Day 1 recap

– Topic 3: Implementing a Multi-Agent AI Workflow (Labs 6–8)

– Lunch Break · Digital Attendance (PM)

– Revision · Course Feedback and TRAQOM Survey

– Digital Attendance (Assessment) · Final Assessment (WA + PP)

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL ACHIEVE

Learning Outcomes

1

LO1 · Evaluate LLM AI models

Identify the strengths and limitations of LLM AI models by running, measuring and comparing them.

2

LO2 · Analyze RAG algorithms

Analyze Retrieval-Augmented Generation algorithms — embeddings, chunking, similarity search — to improve efficiency.

3

LO3 · Assess multi-agent feasibility

Assess the feasibility of implementing multi-agent AI applications with evidence: cost, latency, guardrails and readiness checks.

© 2026 Tertiary Infotech Academy Pte Ltd

THREE TOPICS · 8 HANDS-ON LABS

Course Outline

1

Topic 1 — Introduction to LLM AI Orchestration

LLM strengths & limitations · AI agents · AutoGen setup · model clients · function-calling tools · Labs 1–3.

2

Topic 2 — Retrieval-Augmented Generation (RAG)

Embeddings · chunking · vector databases · similarity search · grounded RAG agents · evaluation · Labs 4–5.

3

Topic 3 — Implementing a Multi-Agent AI Workflow

ReAct · human-in-the-loop · round-robin teams · SelectorGroupChat routing · feasibility · Labs 6–8.

© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE YOU START

Briefing for Assessment

1

Do · Clear your desk

Place phones and other materials under the table or on the floor.

2

Don't · No recording

No photos or recording of assessment scripts.

3

Don't · No discussion

Work individually — no discussion during the assessment.

4

Do · Black or blue pen

Use a black or blue pen for hard-copy assessments.

5

Don't · No correction fluid

No liquid paper or correction tape may be used.

6

Do · Stop on time

Scripts are collected when time is up.

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 2 · 4:30 PM

Final Assessment

1

Written Assessment (WA)

Short-Answer Questions (SAQ) · 1 hour · open book. Tests the underpinning knowledge in these slides.

2

Practical Performance (PP)

Hands-on AutoGen build tasks · 1 hour · open book. Tests the ability you built in the labs.

3

Open book

You may use the course slides, the Learner Guide and approved materials only.

4

Eligibility

A minimum of 75% attendance is required to be eligible for assessment and funding.

5

Result

You are assessed as Competent (C) or Not Yet Competent (NYC) on each instrument.

6

Appeals

An appeal process is available if you wish to contest an assessment outcome.

© 2026 Tertiary Infotech Academy Pte Ltd

ON ASSESSMENT DAY

Assessment Flow

1

TRAQOM survey

Scan the QR code on the…

2

Digital attendance

Scan the SSG assessment QR

3

Sit WA then PP

Open book · 1 hour each

4

Submit on the LMS

Upload your completed…

5

Sign the record

Sign the Assessment…

REMEMBER

All five steps are mandatory for WSQ funding — missing the digital attendance or the TRAQOM survey can invalidate your claim.

© 2026 Tertiary Infotech Academy Pte Ltd

WSQ FUNDING

Criteria for Funding

1

Attendance

A minimum attendance rate of 75%, based on the SSG Digital Attendance record.

2

Assessment

Complete both assessment components and be assessed as 'Competent'.

3

Digital attendance

Scan the SSG QR code for AM, PM and Assessment on every training day.

4

TRAQOM survey

Complete the mandatory TRAQOM course feedback survey on the LMS.

© 2026 Tertiary Infotech Academy Pte Ltd

PRACTICE EXAM · exams.tertiaryinfotech.com

Practice Exams — Sharpen Your Readiness

1

Practice portal

https://exams.tertiaryinfotech.com/practice-exams — free Tertiary Infotech practice exams.

2

Why practise

Timed question practice builds the recall you need for the open-book WA (SAQ).

3

How to use it

Create a free account, pick an AI/agents practice exam, and review every explanation.

4

Best preparation

Redo the 8 labs from a blank file — the PP mirrors what you built in class.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB ENVIRONMENT · labs/lab-00-prelab-uv-setup

Set Up Before We Start (Pre-Lab)

1

Install uv

curl -LsSf https://astral.sh/uv/install.sh | sh (or the PowerShell one-liner on Windows).

2

Clone the labs

git clone the course repository from GitHub — the link is on your LMS course page.

3

Sync the environment

cd labs && uv sync — installs Python, AutoGen AgentChat, Extensions and ChromaDB.

4

Set your API key

cp .env.example .env and paste the OPENAI_API_KEY the trainer provides.

5

Verify

uv run python -c "import autogen_agentchat" prints no error.

6

Editor

Use VS Code or any editor you are comfortable with.

© 2026 Tertiary Infotech Academy Pte Ltd

CORE CONCEPTS

From Large Language Models to Multi-Agent AI

© 2026 Tertiary Infotech Academy Pte Ltd

What is a Large Language Model?

1

A model of language

An LLM such as GPT-4o, Claude or Gemini is trained to understand and generate human language.

2

Why 'large'

Large in training data and parameters — hundreds of billions of learned weights.

3

Autoregressive

It predicts the next token, one token at a time, conditioned on everything written so far.

4

Trained then inferred

Training builds the model from a huge corpus; inference is the prompt-and-response you use every day.

© 2026 Tertiary Infotech Academy Pte Ltd

AUTOREGRESSIVE GENERATION

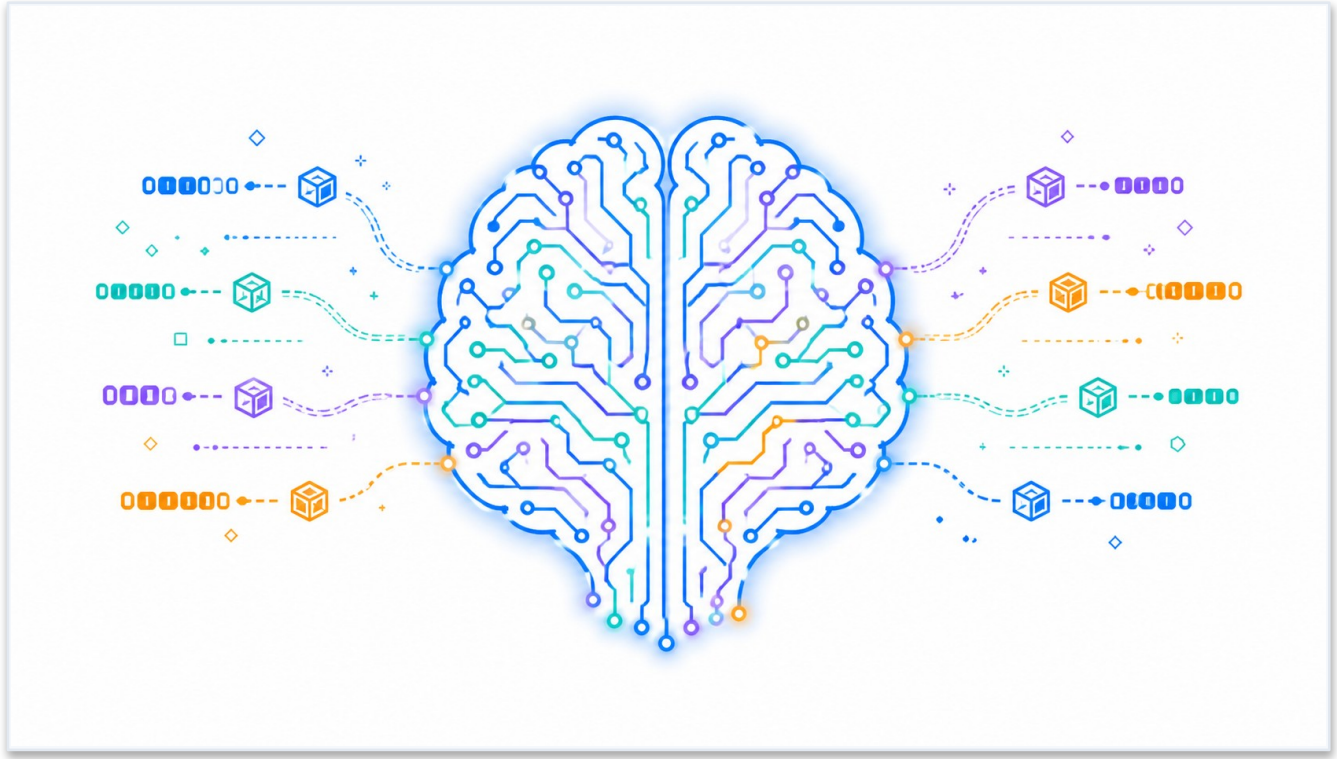
How an LLM Produces an Answer

1. Your prompt is broken into tokens
2. The model attends over the full context
3. It predicts the most likely next token
4. The token is appended and the loop repeats
5. Generation stops at a stop token or limit

© 2026 Tertiary Infotech Academy Pte Ltd

LO1 · THE FOUNDATION

The Engine of Every Agent — the LLM



THE IDEA

Everything in this course sits on top of one capability: a model that turns context into the most plausible next token. Orchestration is the discipline of making that capability reliable.

© 2026 Tertiary Infotech Academy Pte Ltd

LLM Strengths vs Limitations

What LLMs do well

- Strengths
  - Fluent generation, summarisation and translation
  - Broad general knowledge from pre-training
  - In-context learning from a few examples
  - Function calling for structured actions

What you must manage

- Limitations
  - Hallucination — confident but wrong output
  - Knowledge cut-off — no events after training
  - Stateless — no memory between calls
  - Non-deterministic at temperature > 0 · context limits · cost

© 2026 Tertiary Infotech Academy Pte Ltd

LO1 · MEASURABLE AXES

How to Evaluate an LLM for Your Application

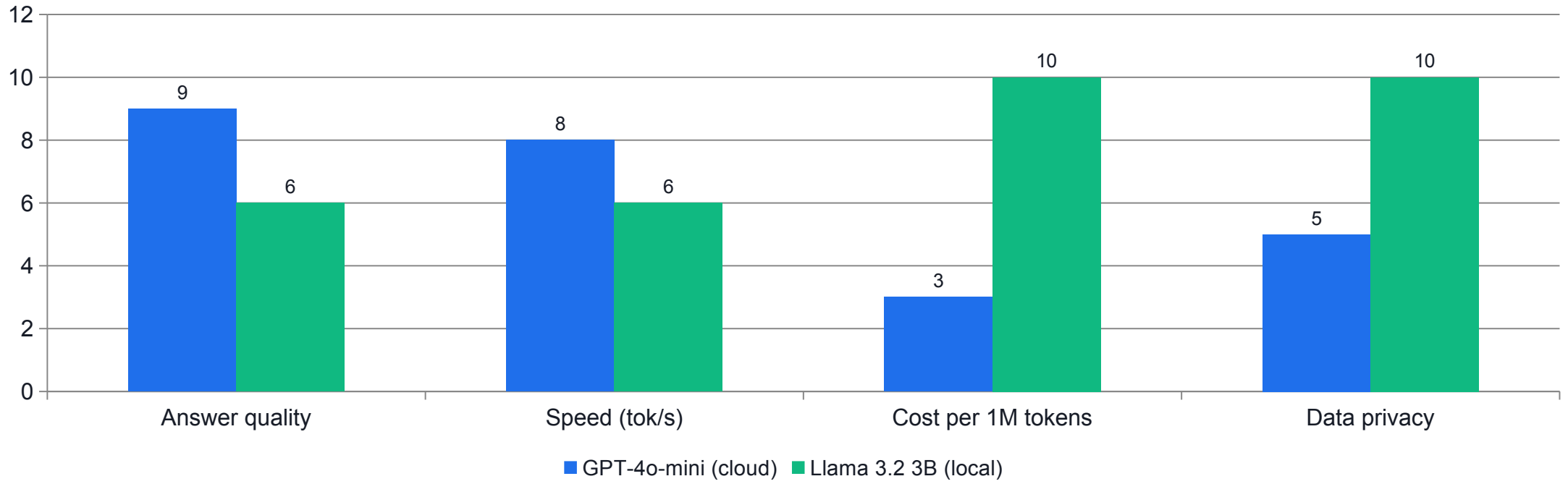
Axis	What you measure	How to measure it	Seen in lab
Quality	Task accuracy, groundedness	Gold questions, human review	Labs 1, 5
Latency	Seconds per response	Timing wrapper around the call	Lab 2
Cost	Tokens in + out per call	response.usage token counts	Lab 1
Repeatability	Same input → same output?	Run twice at temperature 0	Lab 1
Privacy	Where does data go?	Cloud API vs local Ollama model	Lab 2

WHEN IT MATTERS Model choice is an engineering decision made with numbers — not a brand preference.

© 2026 Tertiary Infotech Academy Pte Ltd

LO1 · INDICATIVE 1–10 SCORES

Cloud vs Local Model — the Trade-off



WHAT THE DATA SHOWS

Indicative teaching scores, not benchmarks: the cloud model wins on quality; the local model wins on cost and privacy. Lab 2 measures this trade-off on your own machine.

© 2026 Tertiary Infotech Academy Pte Ltd

WHY AGENTS

An LLM answers. An agent acts.

An AI agent wraps the model with a role, memory and tools — so it can complete tasks, not just produce text.

© 2026 Tertiary Infotech Academy Pte Ltd

THE SIX INGREDIENTS

What Makes a System 'Agentic'?

1

Reasoning

The model plans a sequence of steps instead of answering in one shot.

2

Tool use

It calls functions and APIs to fetch live data or take real actions.

3

Memory

Conversation context lets it carry state across many turns.

4

Autonomy

It decides which tool to call — or which specialist agent should act next.

5

Collaboration

Multiple specialised agents divide the work and hand off between themselves.

6

Guardrails

System messages, termination conditions and human approval constrain what it may do.

© 2026 Tertiary Infotech Academy Pte Ltd

THE PATTERN BEHIND EVERY AGENT

The Agentic Loop — Reason, Act, Observe



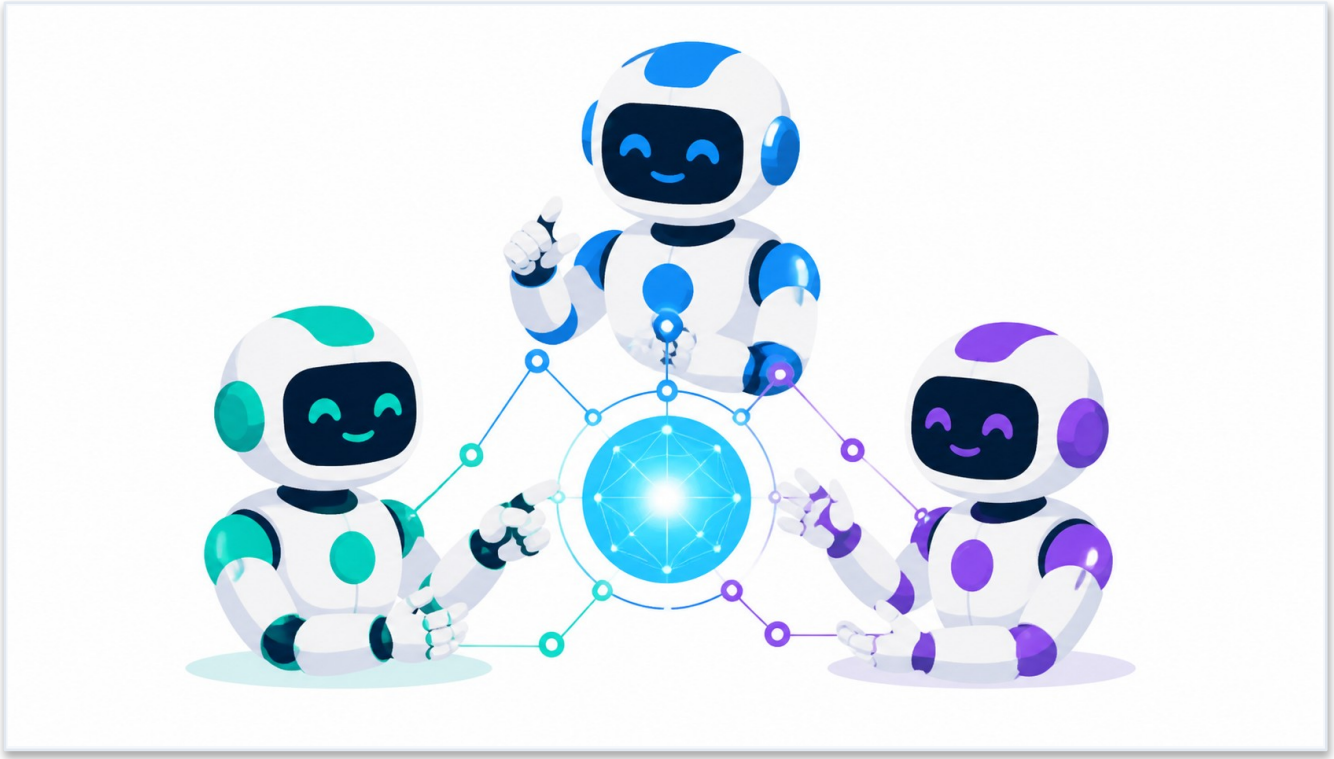
WHY IT MATTERS

The loop is what separates an agent from a chatbot: it can take several actions before answering, and it decides how many.

© 2026 Tertiary Infotech Academy Pte Ltd

THE BIG IDEA

Multi-Agent Systems — Specialists That Collaborate



THE IDEA

One agent with twenty jobs fails quietly; several agents with one job each can be tested, routed and trusted. Dividing work across specialists is the core idea of this course.

© 2026 Tertiary Infotech Academy Pte Ltd

LLM vs Single Agent vs Multi-Agent System

Dimension	Plain LLM	Single Agent	Multi-Agent System
What it does	Generates text	Completes a task	Divides work across specialists
Memory	None — stateless	Conversation context	Shared team conversation
Tools	None	Its own tool set	Each agent has its own tools
Best for	Drafting, summarising	One well-defined job	Broad domains, varied requests
Main risk	Hallucination	Overloaded instruction	Wrong routing / endless loops

WHEN IT MATTERS Add agents only when one instruction can no longer describe the job clearly — complexity you don't need is complexity you must still debug.

© 2026 Tertiary Infotech Academy Pte Ltd

WHY NOW

From RPA to Agentic Process Automation

Yesterday

- RPA — Robotic Process Automation
  - Scripted clicks and keystrokes
  - Breaks when the screen changes
  - No understanding of content
  - Good for stable, repetitive flows

Today

- APA — Agentic Process Automation
  - Agents reason about goals, not scripts
  - Adapts: reads content, decides, calls tools
  - Handles exceptions and hands off to humans
  - Good for knowledge work with variation

© 2026 Tertiary Infotech Academy Pte Ltd

Industry Use Cases of Multi-Agent AI

Customer Support

- L1 triage agent classifies tickets
- Knowledge agent drafts grounded replies
- Escalation agent routes to humans
- 24/7 first response, faster resolution

Finance & Operations

- Research + analysis agent pairs
- Report writer with reviewer agent
- Compliance checks with citations
- Human approval before anything ships

Software & Data

- Coder + tester agent loops
- Data-pipeline monitoring agents
- Documentation writers with critics
- Feasibility checks before rollout

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · KNOW YOUR OPTIONS

Multi-Agent Frameworks — the Landscape

Framework	Orchestration style	Strengths	Best fit
AutoGen	Conversation-driven teams	Teams, tools, human-in-loop, Studio UI	This course — flexible agent teams
LangGraph	Explicit state graph	Deterministic flows, persistence	Regulated, auditable pipelines
CrewAI	Role/task crews	Fast role-based prototyping	Content and research crews
Semantic Kernel	Skills + planners	.NET/enterprise integration	Microsoft-stack enterprises

WHEN IT MATTERS The concepts transfer: agents, tools, memory and termination exist in every framework — AutoGen is our vehicle for learning them.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 1 PREVIEW

What is AutoGen?

1

Microsoft open source

A framework for building multi-agent AI applications, maintained by Microsoft Research.

2

Conversation-first

Agents collaborate by exchanging messages in shared team conversations.

3

Batteries included

Model clients, tools, teams, termination conditions and human-in-the-loop out of the box.

4

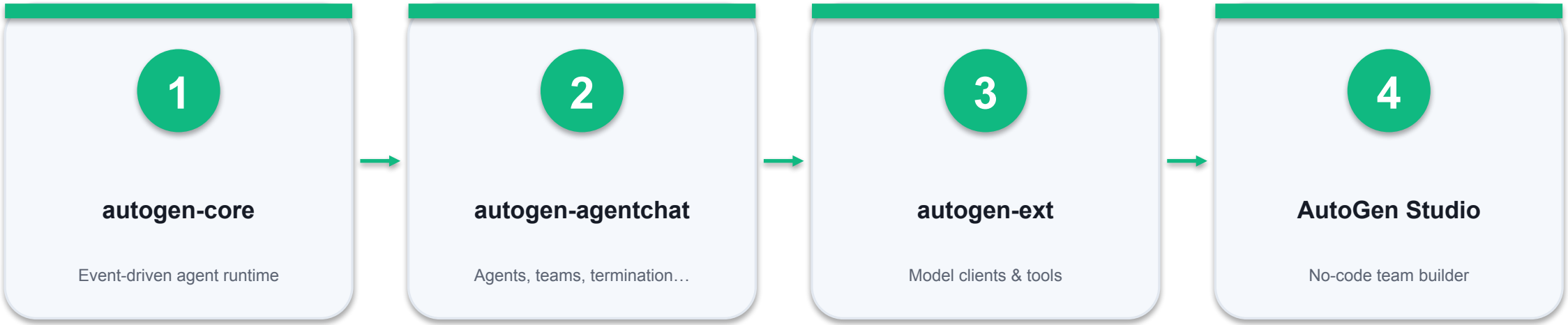
Production path

The same concepts scale from a script to AutoGen Studio to the event-driven Core runtime.

© 2026 Tertiary Infotech Academy Pte Ltd

FOUR LAYERS, ONE FRAMEWORK

The AutoGen Stack



WHERE WE WORK

This course lives in agentchat + ext — the task-driven layer most applications use. Studio gives the same power without code.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · WORKED EXAMPLE

Reading an AutoGen Agent Definition

Every agent in this course is built from these same fields — learn to read them once.

```
from autogen_agentchat.agents import AssistantAgent
from autogen_ext.models.openai import (
    OpenAIChatCompletionClient,
)

client = OpenAIChatCompletionClient(
    model='gpt-4o-mini',
)
agent = AssistantAgent(
    name='ops_assistant',
    model_client=client,
    system_message='You are ...',
    tools=[lookup_ticket],
)
```

model_client

Which LLM runs the agent — swap OpenAI for Azure, Gemini or Ollama here.

name

The identifier used in team conversations and routing.

system_message

The role, rules and refusals. Most behaviour lives here.

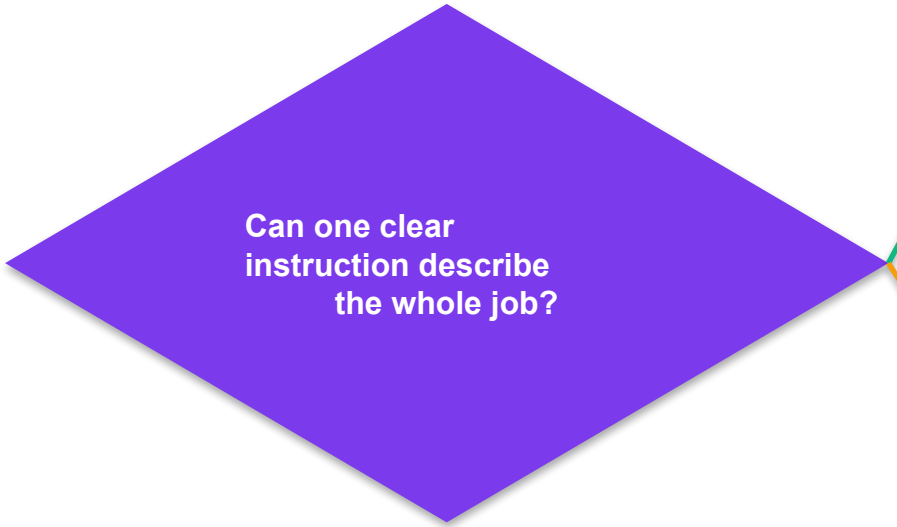
tools

Typed Python functions the agent may call — its hands.

© 2026 Tertiary Infotech Academy Pte Ltd

LO3 · DESIGN DECISION

Which Agent Pattern Should You Use?



YES — use a single agent

Give it the tools it needs. Simpler to build, cheaper to run and far easier to debug. Most business tasks stop here.

NO — use a team

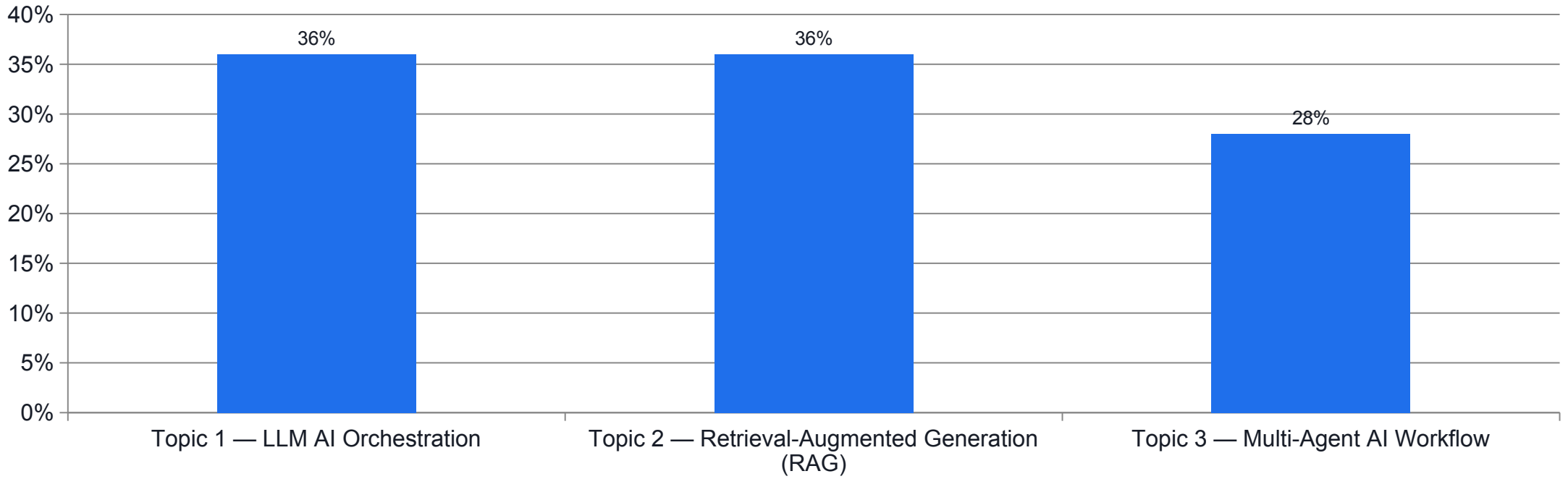
Split by specialism. RoundRobinGroupChat when the order is fixed; SelectorGroupChat when an LLM should route dynamically.

A common mistake is starting multi-agent. Start single, and split only when the instruction starts listing unrelated responsibilities.

© 2026 Tertiary Infotech Academy Pte Ltd

LESSON PLAN · TIME ALLOCATION

Where Course Time Goes — Topic Weighting



WHAT THE DATA SHOWS

Topics 1 and 2 build the orchestration and RAG foundations; Topic 3 assembles them into multi-agent workflows — the skill the practical assessment weighs most.

© 2026 Tertiary Infotech Academy Pte Ltd

01

TOPIC 01

Introduction to LLM AI Orchestration

LLM strengths & limitations · AI agents · AutoGen setup · model clients · tools

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC WEIGHTING 36%

Key Concepts — Introduction to LLM AI Orchestration

1. An LLM generates text token by token from a prompt; it is stateless, has a knowledge cut-off, and can hallucinate — the limitations that orchestration exists to manage.

2. Strengths and limitations are evaluated on measurable axes: quality, latency, cost per call, context length, privacy and repeatability (temperature).

3. An AI agent wraps an LLM with a role (system message), memory of the conversation, and tools — so it can act, not just answer.

4. AutoGen is Microsoft's open-source multi-agent framework: Core (event-driven runtime), AgentChat (task-driven agents & teams), Extensions (model clients, tools) and Studio (no-code UI).

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC WEIGHTING 36%

Key Concepts — Introduction to LLM AI Orchestration (continued)

1. A model client (OpenAIChatCompletionClient) abstracts the provider — the same agent code runs on OpenAI, Azure, Gemini, or a local Ollama model.

2. Function calling turns typed Python functions into tools the model can request; AutoGen's FunctionTool generates the JSON schema from the signature and docstring.

3. Structured output (output_content_type with a Pydantic model) converts free text into validated fields that downstream code can branch on.

4. Async execution (asyncio, run/run_stream, the Console UI) is how AgentChat streams events and scales beyond one blocking call.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · ORCHESTRATION

LLM AI Orchestration — What and Why

1

Definition

Coordinating models, prompts, tools, data and control flow into one dependable application.

2

Beyond one prompt

Real apps chain calls, branch on results, retry failures and log everything.

3

The orchestrator's job

Decide what runs, in what order, with what context — and when to stop.

4

Frameworks help

AutoGen supplies the loop, message types, tool plumbing and termination logic.

© 2026 Tertiary Infotech Academy Pte Ltd

Running a Local LLM with Ollama

1

Why local

Privacy (data never leaves the machine), zero per-call cost, offline capability.

2

Install & pull

ollama pull llama3.2 downloads a quantised open model to your laptop.

3

OpenAI-compatible

Ollama serves http://localhost:11434/v1 — the SAME client code works against it.

4

The catch

Smaller local models trade answer quality and speed for privacy and cost — measure it in Lab 2.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · WORKED EXAMPLE

Building a Minimal LLM App

Lab 1's llm_app.py — the irreducible core of every LLM application.

```
from openai import OpenAI

client = OpenAI(api_key=KEY)
resp = client.chat.completions.create(
    model='gpt-4o-mini',
    temperature=0.7,
    messages=[
        {'role':'system','content':'...'},
        {'role':'user','content':'...'},
    ],
)
print(resp.choices[0].message.content)
print(resp.usage.total_tokens)
```

messages

The whole conversation is sent every call — the model itself is stateless.

temperature

0 = repeatable, higher = varied. Your first debugging lever.

usage

Token counts are your cost meter — log them from day one.

choices[0]

The reply. Everything else in this course wraps this call.

© 2026 Tertiary Infotech Academy Pte Ltd

Debugging an LLM App

1

Log the full prompt

Most 'model bugs' are prompt bugs — print exactly what was sent.

2

Fix the temperature

Set temperature=0 while debugging so runs are comparable.

3

Watch the tokens

Truncated answers usually mean a max-token or context limit was hit.

4

Isolate the layer

Test the raw API call first (Lab 1), then the agent wrapper — never both at once.

5

Read the events

AutoGen streams every message and tool call — the Console UI is your trace.

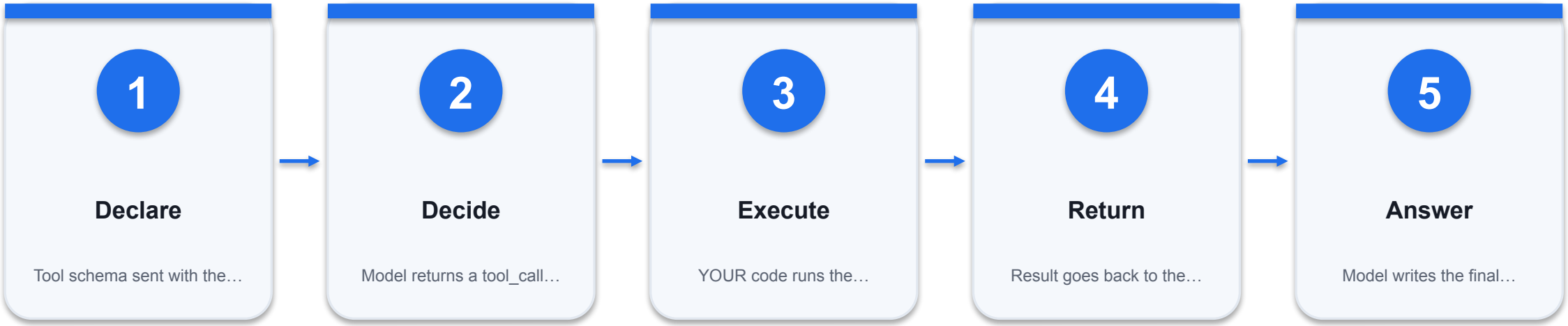
6

Reproduce cheaply

Keep a failing prompt as a test case you can re-run after every change.

© 2026 Tertiary Infotech Academy Pte Ltd

How Function Calling Works



THE KEY INSIGHT

The model never executes anything — it only asks. Your code stays in control of every action, which is where safety lives.

© 2026 Tertiary Infotech Academy Pte Ltd

Three Kinds of Tools in AutoGen

Custom FunctionTool

- Any typed Python function
- Schema generated from the signature
- Docstring becomes the description
- Labs 3–5, 8 use these

Built-in / Extensions

- autogen-ext ships ready-made tools
- HTTP requests, code execution, MCP
- Install-and-go integrations
- Explore: autogen_ext.tools

Third-party APIs

- Wrap any REST API in a function
- Weather, search (Tavily), arXiv…
- Keep keys in .env, never in code
- Same FunctionTool pattern

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Introduction to LLM AI Orchestration

Lab 1

- Set Up AutoGen and Run Your First Agent

Lab 2

- Configure Model Clients and Structured Output

Lab 3

- Equip an Agent with Tools

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Set Up AutoGen and Run Your First Agent

LAB 1

LO1 — evaluate an LLM by running it directly, then as an AutoGen agent.

Verify the uv lab environment from the Pre-Lab, call an LLM directly with the OpenAI SDK, then wrap the same model in an AutoGen AssistantAgent and stream its reply with the Console UI.

YOU'LL BUILD

A working AutoGen environment plus two runnable scripts: a plain LLM app and your first agent.

TOOLCHAIN

uv, Python 3.10+, OpenAI SDK, AutoGen AgentChat, Console UI

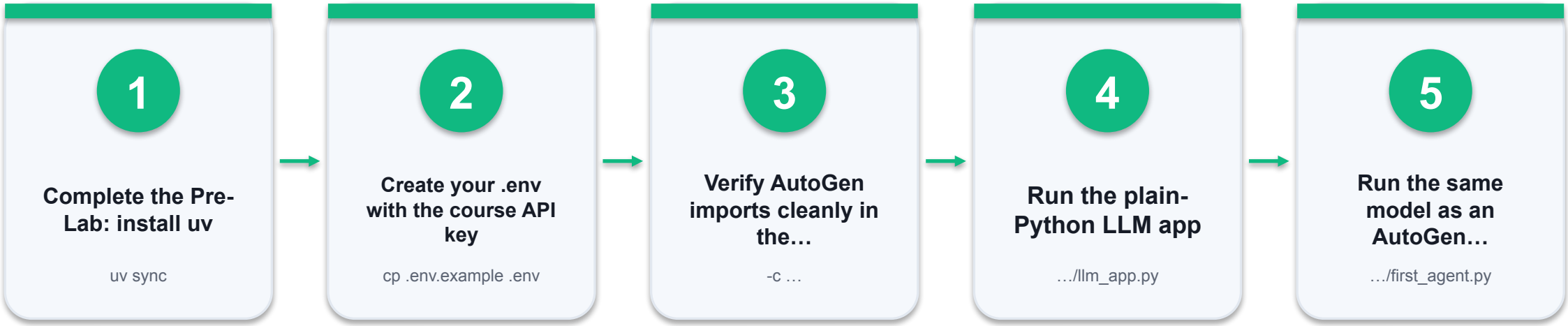
DONE WHEN

Both scripts print a model reply with no traceback, and you can state one strength (fluent answers) and one limitation (stateless, non-repeatable at temperature 0.7) you just observed.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · PROCESS MAP

How Lab 1 Runs



THE POINT

LO1 — evaluate an LLM by running it directly, then as an AutoGen agent.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · PROCEDURE

Set Up AutoGen and Run Your First Agent — Part 1/2

1

Complete the Pre-Lab: install uv, then sync the lab environment

```
$ uv sync
```

2

Create your .env with the course API key (never commit it)

```
$ cp .env.example .env
```

3

Verify AutoGen imports cleanly in the environment

```
$ uv run python -c "import autogen_agentchat; print('OK')"
```

4

Run the plain-Python LLM app and note the reply and token count

```
$ uv run python lab-01-first-autogen-agent/llm_app.py
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · PROCEDURE

Set Up AutoGen and Run Your First Agent — Part 2/2

5

Run the same model as an AutoGen AssistantAgent with streaming

```
$ uv run python lab-01-first-autogen-agent/first_agent.py
```

6

Set temperature to 0 in llm_app.py and re-run twice to compare repeatability

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · VERIFY

Set Up AutoGen and Run Your First Agent

✅ Expected result

Both scripts print a model reply with no traceback, and you can state one strength (fluent answers) and one limitation (stateless, non-repeatable at temperature 0.7) you just observed.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the OPENAI_API_KEY into labs/.env — no quotes, no spaces.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Configure Model Clients and Structured Output

LAB 2

LO1 — compare cloud vs local models and constrain output to a schema.

Run the same prompt against OpenAI's cloud API and a local Ollama model to compare quality, latency and cost, then force an AutoGen agent to reply in a validated Pydantic TicketTriage schema instead of free text.

YOU'LL BUILD

A model-comparison script with timings, and a triage agent that returns validated JSON fields.

TOOLCHAIN

OpenAIChatCompletionClient, Ollama (optional), Pydantic, output_content_type

DONE WHEN

The triage agent returns a valid TicketTriage object with priority P1 for the outage ticket; you can name the trade-offs (latency, cost, privacy, quality) between the cloud and local model.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · PROCESS MAP

How Lab 2 Runs

1. **Run the model-client comparison**
   …/model_clients.py
2. **Optional: install Ollama**
   ollama pull llama3.2
3. **Read the TicketTriage Pydantic schema — the…**
4. **Run the structured-output triage agent on…**
   …/structured_agent.py
5. **Change the ticket text to a P4 request**

THE POINT

LO1 — compare cloud vs local models and constrain output to a schema.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · PROCEDURE

Configure Model Clients and Structured Output — Part 1/2

1. **Run the model-client comparison (OpenAI cloud, with timing)**

```
$ uv run python lab-02-model-clients-structured-output/model_clients.py
```

2. **Optional: install Ollama and pull a local model, then re-run to compare**

```
$ ollama pull llama3.2
```

3. Read the TicketTriage Pydantic schema — the contract the agent must obey

4. **Run the structured-output triage agent on the outage ticket**

```
$ uv run python lab-02-model-clients-structured-output/structured_agent.py
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · PROCEDURE

Configure Model Clients and Structured Output — Part 2/2

5 Change the ticket text to a P4 request and re-run — watch the fields change

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · VERIFY

Configure Model Clients and Structured Output

✅ Expected result

The triage agent returns a valid TicketTriage object with priority P1 for the outage ticket; you can name the trade-offs (latency, cost, privacy, quality) between the cloud and local model.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the OPENAI_API_KEY into labs/.env — no quotes, no spaces.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Equip an Agent with Tools

LAB 3

LO1 — extend an LLM's limits with function calling and FunctionTool.

Do raw OpenAI function calling by hand against the synthetic ticket dataset, then let AutoGen's FunctionTool generate the schema, execute the calls and reflect on the results with two read-only ticket tools.

YOU'LL BUILD

A hand-rolled function-calling round trip, and a help-desk agent with lookup_ticket and count_open_tickets tools.

TOOLCHAIN

OpenAI function calling, FunctionTool, reflect_on_tool_use, tickets.json mock data

DONE WHEN

The agent reports the real subject of TT-1005 and the correct count of open P2 tickets — every fact traceable to a tool result, none invented.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · PROCESS MAP

How Lab 3 Runs

1. **Inspect the synthetic ticket data the tools…**
   cat data/tickets.json
2. **Run raw function calling: schema →…**
   …/function_calling.py
3. **Read tool_agent.py — one typed Python…**
4. **Run the AutoGen tool agent**
   …/tool_agent.py
5. **Ask a question needing BOTH tools**

THE POINT

LO1 — extend an LLM's limits with function calling and FunctionTool.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · PROCEDURE

Equip an Agent with Tools — Part 1/2

1

Inspect the synthetic ticket data the tools will read

```
$ cat data/tickets.json
```

2

Run raw function calling: schema → tool_call → execute → final answer

```
$ uv run python lab-03-agent-tools/function_calling.py
```

3

Read tool_agent.py — one typed Python function becomes one FunctionTool line

4

Run the AutoGen tool agent and watch ToolCallRequestEvent / ToolCallExecutionEvent

```
$ uv run python lab-03-agent-tools/tool_agent.py
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · PROCEDURE

Equip an Agent with Tools — Part 2/2

5 Ask a question needing BOTH tools and confirm two tool calls are made

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · VERIFY

Equip an Agent with Tools

✅ Expected result

The agent reports the real subject of TT-1005 and the correct count of open P2 tickets — every fact traceable to a tool result, none invented.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the OPENAI_API_KEY into labs/.env — no quotes, no spaces.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Introduction to LLM AI Orchestration

- You can now: evaluate an LLM by running it directly, then as an AutoGen agent.
- You can now: compare cloud vs local models and constrain output to a schema.
- You can now: extend an LLM's limits with function calling and FunctionTool.

© 2026 Tertiary Infotech Academy Pte Ltd

02

TOPIC 02

Retrieval-Augmented Generation (RAG)

Embeddings · chunking · vector database · similarity search · grounded agents · evaluation

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC WEIGHTING 36%

Key Concepts — Retrieval-Augmented Generation (RAG)

1. RAG lets an LLM answer from knowledge outside its training data by retrieving relevant text and stuffing it into the prompt — fresher answers, fewer hallucinations, source citations.

2. A text embedding maps text to a vector; semantically similar texts land close together, so meaning becomes measurable as cosine similarity.

3. Chunking decides what one retrievable unit is; chunk boundaries (by section, paragraph or tokens, with overlap) directly change retrieval quality.

4. A vector database (ChromaDB) stores embeddings and answers approximate nearest-neighbour queries in milliseconds.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC WEIGHTING 36%

Key Concepts — Retrieval-Augmented Generation (RAG) (continued)

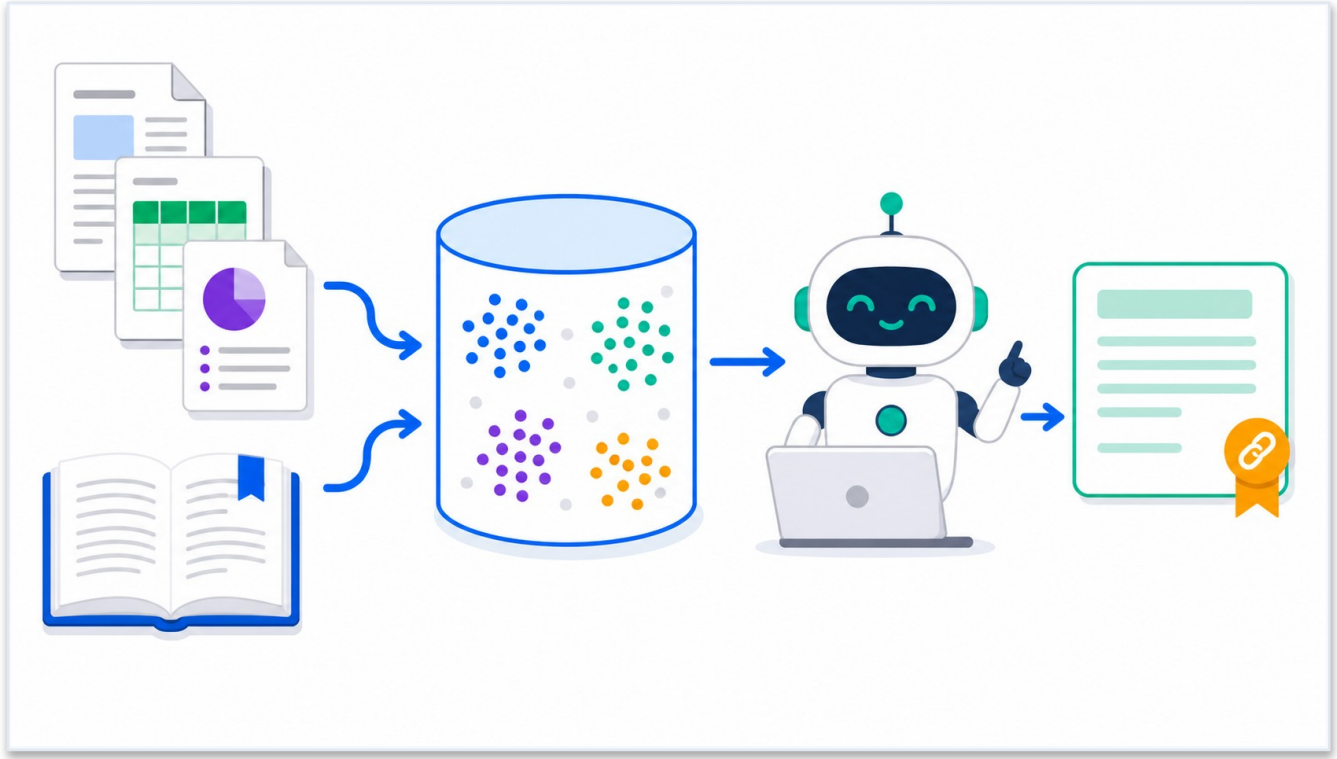
1. The RAG pipeline: load → chunk → embed → store at indexing time; embed query → similarity search → augment prompt → generate at answer time.

2. An agentic RAG agent calls retrieval as a tool, cites source ids, and refuses when evidence is missing — and treats retrieved text as data, never as instructions (prompt-injection defence).

3. RAG efficiency is analysed with numbers: hit@k retrieval accuracy on a gold question set, retrieval latency, chunk count and top-k — change one lever, re-measure.

© 2026 Tertiary Infotech Academy Pte Ltd

Retrieval-Augmented Generation at a Glance



THE IDEA

Your documents are chunked and embedded into a vector database; at question time the most similar chunks are retrieved and given to the model as evidence — answers become current, grounded and citable.

© 2026 Tertiary Infotech Academy Pte Ltd

Why RAG?

1

Fresh knowledge

Answers come from YOUR documents, not the model's training cut-off.

2

Fewer hallucinations

The model quotes retrieved evidence instead of inventing facts.

3

Citations

Every answer can name its source id — auditable and trustworthy.

4

Cheaper than fine-tuning

Update the index, not the model — new policy PDF, same LLM.

5

Access control

Retrieval can respect who is allowed to see which document.

6

Smaller prompts

Only the relevant chunks travel — not the whole handbook.

© 2026 Tertiary Infotech Academy Pte Ltd

Text Embeddings — Meaning as Numbers

1

What

An embedding model maps text to a vector of hundreds of numbers.

2

Property

Semantically similar texts land close together in that vector space.

3

Similarity

Closeness is measured with cosine similarity (or distance = 1 − similarity).

4

In our labs

ChromaDB embeds locally with all-MiniLM-L6-v2 — no API key, no data leaving your laptop.

© 2026 Tertiary Infotech Academy Pte Ltd

Chunking Strategies — the First RAG Design Decision

Strategy	How it splits	Pros	Cons
By section header	One policy / heading per chunk	Coherent, self-contained units	Sections can be long
Fixed tokens + overlap	e.g. 500 tokens, 50 overlap	Uniform size, simple	Cuts mid-thought
By paragraph	Every paragraph separate	Fine-grained retrieval	Loses surrounding context
Semantic chunking	Split where topic shifts	Best coherence	Extra compute to build

WHEN IT MATTERS Lab 4 chunks the handbook by '## ' headers so each chunk is one complete policy — re-chunk and re-measure to see why boundaries matter.

© 2026 Tertiary Infotech Academy Pte Ltd

Vector Databases and Similarity Search

1

Store

A vector DB (ChromaDB, FAISS, Pinecone, pgvector) persists embeddings with ids and metadata.

2

Search

A query is embedded and the nearest neighbours are returned in milliseconds.

3

Top-k

You choose how many chunks come back — more recall vs more noise and tokens.

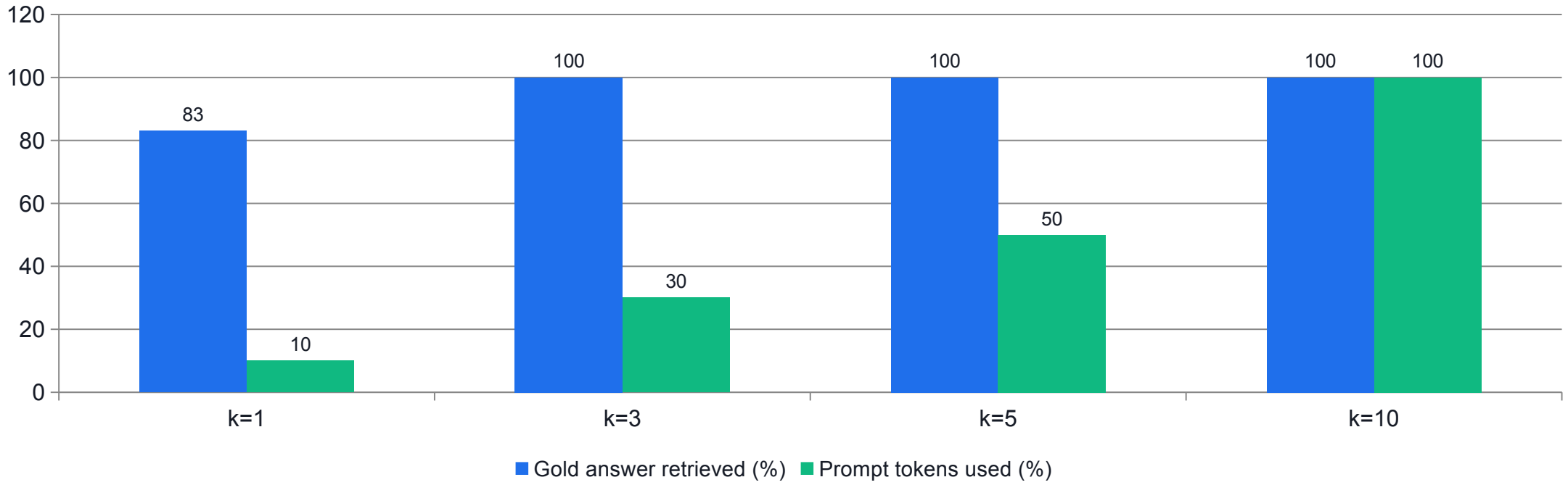
4

Metadata

Source ids (POL-001) ride along with each chunk — that's what citations are made of.

© 2026 Tertiary Infotech Academy Pte Ltd

Retrieval Quality vs Top-k — Measured, Not Guessed



WHAT THE DATA SHOWS

Illustrative numbers in the shape you will measure in Lab 5: recall saturates quickly while token cost keeps climbing — the efficient k is the smallest one that still hits your accuracy target.

© 2026 Tertiary Infotech Academy Pte Ltd

Agentic RAG — Retrieval as a Tool

1

Classic RAG

The pipeline always retrieves once, then answers — fixed behaviour.

2

Agentic RAG

The AGENT decides when to retrieve, with what query, and whether to retry with a better one.

3

Grounding rules

Always retrieve first · cite source ids · refuse when evidence is missing.

4

Injection defence

Retrieved text is DATA — an agent must never obey instructions found inside documents.

© 2026 Tertiary Infotech Academy Pte Ltd

Evaluating RAG Efficiency

1

hit@k

Does the gold source appear in the top-k results? Measured over a stable question set.

2

Latency

Milliseconds per retrieval — the index must be faster than the model.

3

Groundedness

Is every claim in the answer supported by a retrieved chunk?

4

The method

Change ONE lever (chunking, top-k, embedder) → re-run the gold set → compare numbers.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Retrieval-Augmented Generation (RAG)

Lab 4

- Build a Knowledge Index with Embeddings and a Vector Database

Lab 5

- Build and Evaluate a Grounded RAG Agent

—

- —

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Build a Knowledge Index with Embeddings and a Vector Database

LAB 4

LO2 — build the retrieval half of RAG and measure similarity directly.

Chunk the synthetic TechTrend IT handbook into one-policy chunks, embed each chunk locally, store the vectors in ChromaDB, then run similarity searches and inspect which sources return and at what cosine distance.

YOU'LL BUILD

A persistent 10-chunk ChromaDB index of the handbook, a similarity-search console, and a first index-backed agent.

TOOLCHAIN

ChromaDB, local embeddings (all-MiniLM-L6-v2), opsdesk_handbook.md mock data

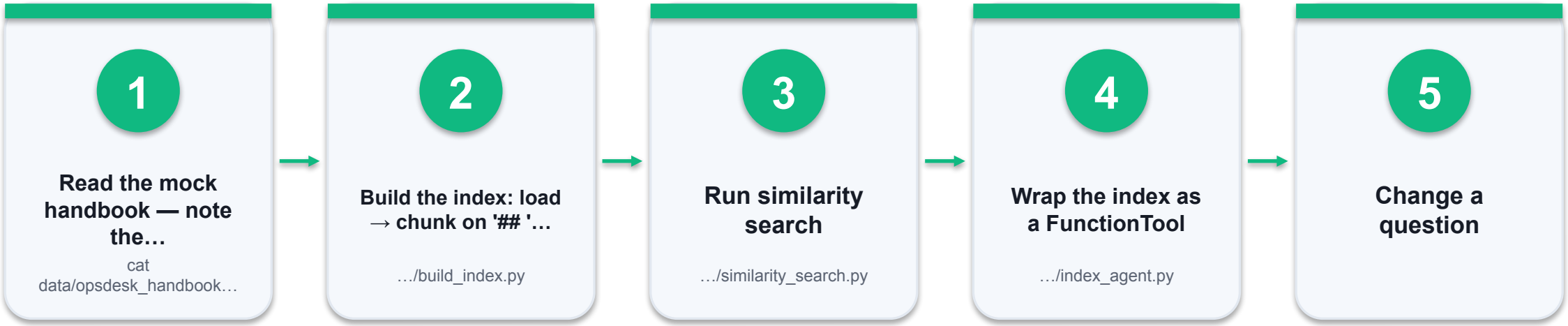
DONE WHEN

build_index.py reports 10 stored chunks; the VPN question returns POL-001 and the P1 question returns PROC-101 as top-1, each with the smallest distance.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · PROCESS MAP

How Lab 4 Runs



THE POINT

LO2 — build the retrieval half of RAG and measure similarity directly.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · PROCEDURE

Build a Knowledge Index with Embeddings and a Vector Database — Part 1/2

1

Read the mock handbook — note the POL-/PROC-/SLA-/SEC- source ids

```
$ cat data/opsdesk_handbook.md
```

2

Build the index: load → chunk on '## ' headers → embed → store

```
$ uv run python lab-04-knowledge-index/build_index.py
```

3

Run similarity search and study the top-3 sources and distances per question

```
$ uv run python lab-04-knowledge-index/similarity_search.py
```

4

Wrap the index as a FunctionTool and let an agent search it

```
$ uv run python lab-04-knowledge-index/index_agent.py
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · PROCEDURE

Build a Knowledge Index with Embeddings and a Vector Database — Part 2/2

5 Change a question and predict the top-1 source before running — check yourself

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · VERIFY

Build a Knowledge Index with Embeddings and a Vector Database

✅ Expected result

build_index.py reports 10 stored chunks; the VPN question returns POL-001 and the P1 question returns PROC-101 as top-1, each with the smallest distance.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the OPENAI_API_KEY into labs/.env — no quotes, no spaces.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Build and Evaluate a Grounded RAG Agent

LAB 5

LO2 — engineer grounding rules, then quantify retrieval efficiency.

Turn the index into a production-shaped RAG agent — mandatory retrieval, source citations, refusal when evidence is missing, prompt-injection defence — then measure hit@1, hit@3 and latency over a gold question set.

YOU'LL BUILD

A grounded rag_agent with a retrieve() tool, plus an evaluation scoreboard for retrieval accuracy and latency.

TOOLCHAIN

AssistantAgent, FunctionTool retrieve(), reflect_on_tool_use, gold question set

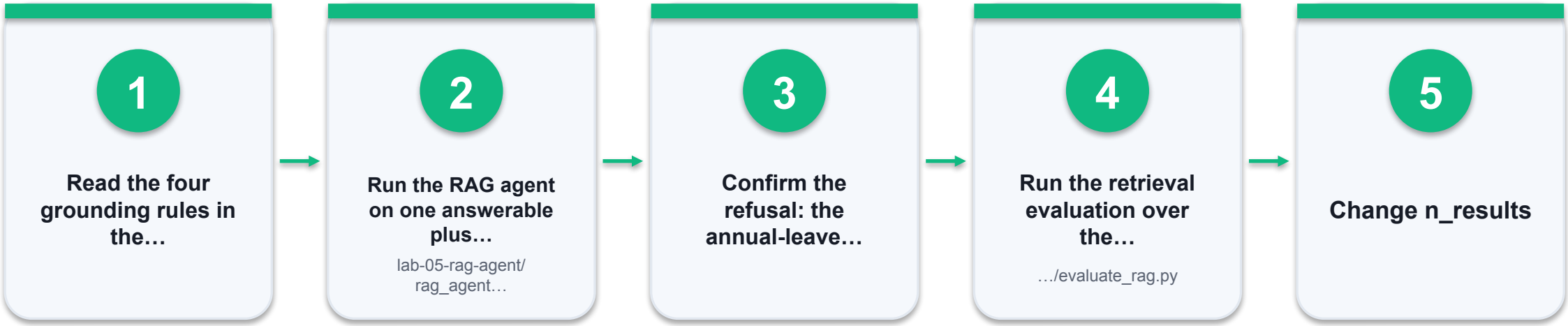
DONE WHEN

The VPN answer carries citations like (POL-001); the annual-leave question is refused; the scoreboard shows hit@1 ≥ 83% with millisecond-level latency.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · PROCESS MAP

How Lab 5 Runs



THE POINT

LO2 — engineer grounding rules, then quantify retrieval efficiency.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · PROCEDURE

Build and Evaluate a Grounded RAG Agent — Part 1/2

1. Read the four grounding rules in the rag_agent system message

2. **Run the RAG agent on one answerable plus one unanswerable question**

```
$ uv run python lab-05-rag-agent/rag_agent.py
```

3. Confirm the refusal: the annual-leave question must NOT be answered from thin air

4. **Run the retrieval evaluation over the 6-question gold set**

```
$ uv run python lab-05-rag-agent/evaluate_rag.py
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · PROCEDURE

Build and Evaluate a Grounded RAG Agent — Part 2/2

5 Change n_results (top-k) and re-run — record how hit@3 and latency move

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · VERIFY

Build and Evaluate a Grounded RAG Agent

✅ Expected result

The VPN answer carries citations like (POL-001); the annual-leave question is refused; the scoreboard shows hit@1 ≥ 83% with millisecond-level latency.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the OPENAI_API_KEY into labs/.env — no quotes, no spaces.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Retrieval-Augmented Generation (RAG)

- You can now: build the retrieval half of RAG and measure similarity directly.
- You can now: engineer grounding rules, then quantify retrieval efficiency.

© 2026 Tertiary Infotech Academy Pte Ltd

03

TOPIC 03

Implementing a Multi-Agent AI Workflow

ReAct · human-in-the-loop · teams · termination · SelectorGroupChat · feasibility

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC WEIGHTING 28%

Key Concepts — Implementing a Multi-Agent AI Workflow

1. ReAct (Reason + Act) is the loop behind every agent: Thought → Action (tool call) → Observation, repeated until the agent can answer.

2. Multi-agent frameworks — AutoGen, LangGraph, CrewAI — differ in orchestration style: conversation-driven teams, explicit state graphs, and role/task crews respectively.

3. A RoundRobinGroupChat team shares one conversation and takes turns in a fixed order — the simplest reliable pattern for writer/reviewer workflows.

4. Termination conditions (TextMentionTermination, MaxMessageTermination, combined with | and &) keep every team run bounded in messages, time and cost.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC WEIGHTING 28%

Key Concepts — Implementing a Multi-Agent AI Workflow (continued)

1. Human-in-the-loop: a UserProxyAgent puts a person inside the team as an approval gate — essential where actions have real-world consequences.

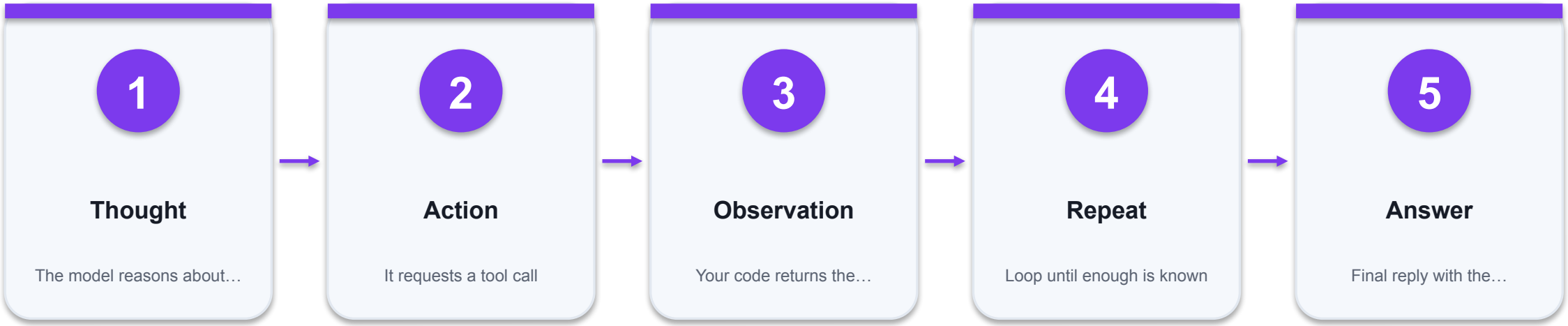
2. SelectorGroupChat uses an LLM selector that reads each agent's description to route the next turn dynamically — specialists get chosen, not scheduled.

3. Feasibility of a multi-agent application is assessed on evidence: task fit, cost and latency per run, failure modes, guardrails, and readiness checks that pass.

4. AutoGen Studio provides a no-code way to prototype the same teams, then export the configuration to code.

© 2026 Tertiary Infotech Academy Pte Ltd

The ReAct Pattern — Reason + Act



WHY IT MATTERS

ReAct is the loop inside AssistantAgent's tool use — Lab 6 builds it from scratch so you can debug it anywhere.

© 2026 Tertiary Infotech Academy Pte Ltd

Conversation Patterns for Agent Teams

1. **Two-agent chat**
   One assistant + one user proxy — the simplest interactive loop.

2. **Round-robin team**
   Fixed turn order in one shared conversation — writer → reviewer → writer.

3. **Selector team**
   An LLM selector reads agent descriptions and routes each turn dynamically.

4. **Swarm / handoff**
   Agents explicitly hand off control with tool calls — fine-grained delegation.

5. **Nested / sequential**
   A team can be a participant inside a bigger workflow.

6. **Human in the loop**
   A UserProxyAgent makes a person one of the participants.

© 2026 Tertiary Infotech Academy Pte Ltd

Termination Conditions — Keeping Teams Bounded

1

TextMentionTermination

Stop when a keyword like APPROVE or TERMINATE appears.

2

MaxMessageTermination

Stop after N messages — the cost/loop safety net every team needs.

3

Combine with | and &

OR stops on either; AND requires both — compose policies from parts.

4

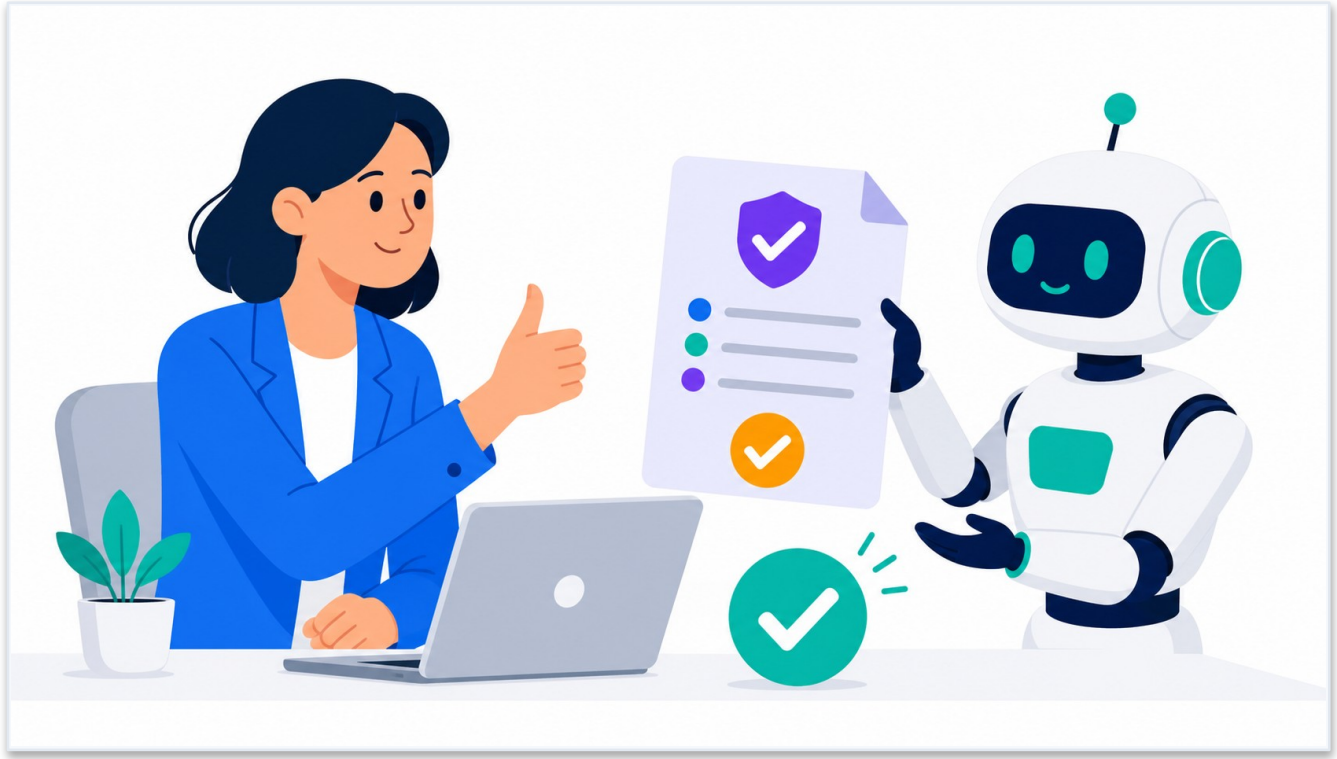
Why it matters

An unbounded team of eager agents can loop — and bill — forever.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · TRUST

Human-in-the-Loop — the Approval Gate



THE IDEA

Where actions have real consequences, a person joins the team: the UserProxyAgent turns your keyboard into an agent, and nothing ships until you type APPROVE.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · WORKED EXAMPLE

Reading a SelectorGroupChat

Lab 8's capstone team — routing is driven by the description fields.

```
team = SelectorGroupChat(
    [planner,
     ticket_specialist,
     policy_specialist],
    model_client=client,
    termination_condition=(
        TextMentionTermination('TERMINATE')
        | MaxMessageTermination(15)
    ),
    allow_repeated_speaker=False,
)
```

participants

The specialists. Each one's description is its job advert.

model_client

The selector LLM that reads descriptions and picks the next speaker.

termination

Keyword OR message cap — bounded by construction.

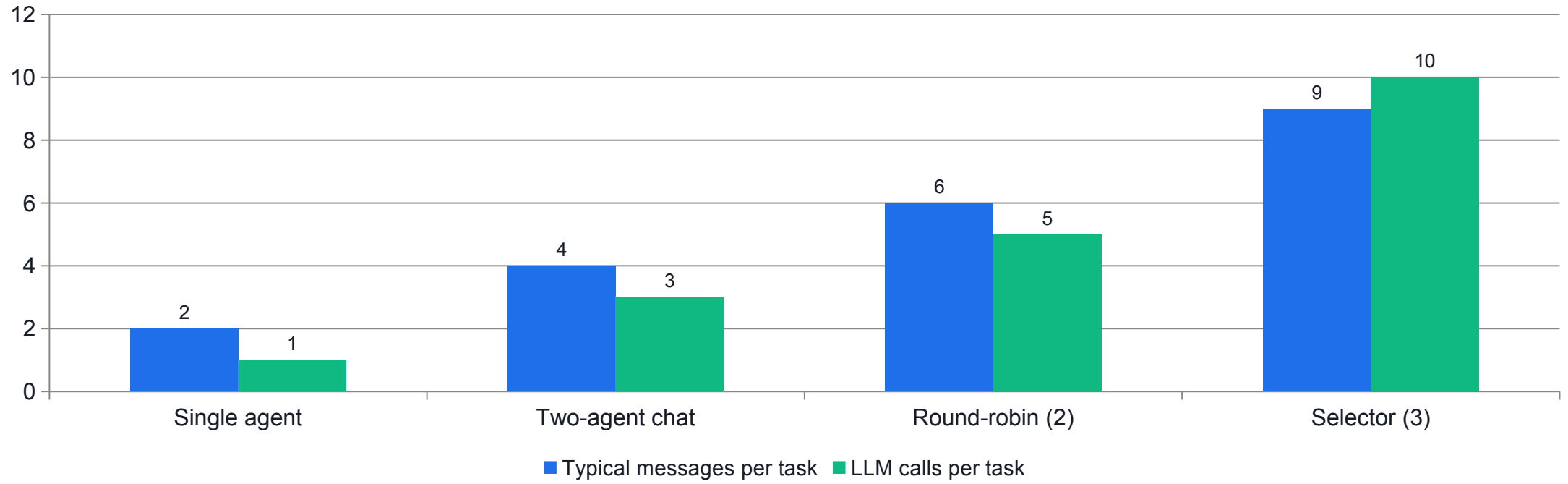
allow_repeated_speaker

Forces variety so one agent cannot monopolise the chat.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · FEASIBILITY NUMBERS

What a Team Run Costs — Messages per Pattern



WHAT THE DATA SHOWS

Indicative counts from the course labs: every extra agent multiplies calls, cost and latency — a multi-agent design must earn its keep, which is exactly what a feasibility assessment checks.

© 2026 Tertiary Infotech Academy Pte Ltd

Assessing Feasibility of a Multi-Agent Application

1 **Task fit**
Does the job really need multiple specialists, or would one instruction do?

2 **Cost & latency**
Messages × model price per run — measured from TaskResult, not guessed.

3 **Failure modes**
Wrong routing, endless loops, hallucinated tool arguments — and their guardrails.

4 **Human oversight**
Where must a person approve? UserProxyAgent puts the gate in the flow.

5 **Readiness checks**
Data present, index healthy, retrieval accurate, secrets out of Git (Lab 8).

6 **Go / no-go**
The verdict is evidence-based: checks pass, costs acceptable, risks gated.

© 2026 Tertiary Infotech Academy Pte Ltd

AutoGen Studio — Teams Without Code

1

What it is

A local web UI for building and testing agent teams visually.

2

Launch

uv run --with autogenstudio autogenstudio ui — then open the browser.

3

Round-trip

Prototype in Studio, export the JSON config, load it from code.

4

When to use it

Demos, quick experiments and non-developer stakeholders.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL DO

Hands-On Labs — Implementing a Multi-Agent AI Workflow

Lab 6

- Implement a ReAct Agent with Human-in-the-Loop

Lab 7

- Orchestrate a Round-Robin Multi-Agent Team

Lab 8

- Route Specialists with SelectorGroupChat (Capstone)

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Implement a ReAct Agent with Human-in-the-Loop

LAB 6

LO3 — expose the ReAct loop, then gate an agent behind human approval.

Hand-roll the Thought → Action → Observation loop so the ReAct pattern is visible, then build an AutoGen team where a UserProxyAgent makes YOU the approval gate before any draft is accepted.

YOU'LL BUILD

A from-scratch ReAct agent over the ticket tools, and an approval workflow that stops only on APPROVE.

TOOLCHAIN

ReAct pattern, UserProxyAgent, input_func, TextMentionTermination, RoundRobinGroupChat

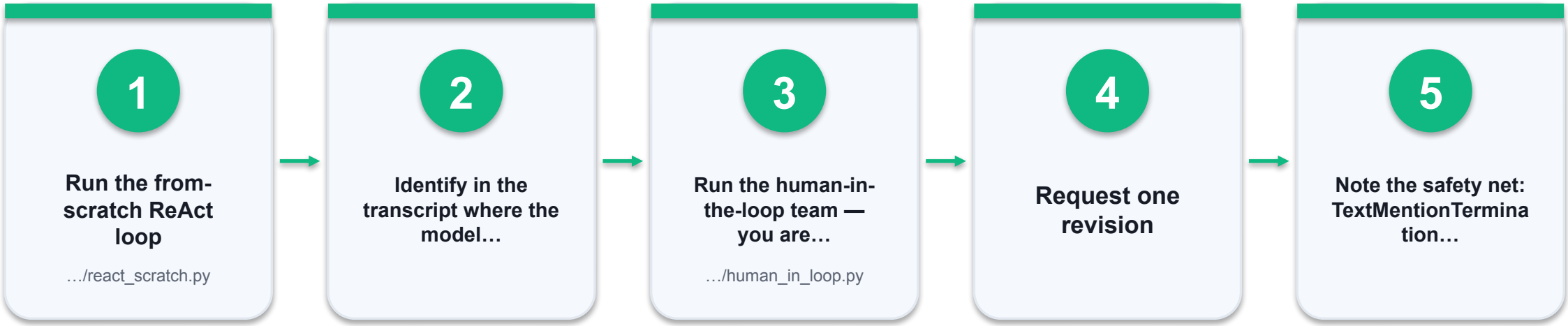
DONE WHEN

The ReAct agent reports the 15-minute P1 SLA target after two tool actions, and the approval team loops on your feedback and stops only when you type APPROVE.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · PROCESS MAP

How Lab 6 Runs



THE POINT

LO3 — expose the ReAct loop, then gate an agent behind human approval.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · PROCEDURE

Implement a ReAct Agent with Human-in-the-Loop — Part 1/2

1. **Run the from-scratch ReAct loop and follow each Thought / Action / Observation**

```
$ uv run python lab-06-react-human-in-loop/react_scratch.py
```

2. Identify in the transcript where the model plans vs where YOUR code acts

3. **Run the human-in-the-loop team — you are the reviewer**

```
$ uv run python lab-06-react-human-in-loop/human_in_loop.py
```

4. Request one revision ('make it more apologetic'), then approve with APPROVE

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · PROCEDURE

Implement a ReAct Agent with Human-in-the-Loop — Part 2/2

5 Note the safety net: TextMentionTermination('APPROVE') | MaxMessageTermination(12)

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · VERIFY

Implement a ReAct Agent with Human-in-the-Loop

✅ Expected result

The ReAct agent reports the 15-minute P1 SLA target after two tool actions, and the approval team loops on your feedback and stops only when you type APPROVE.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the OPENAI_API_KEY into labs/.env — no quotes, no spaces.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Orchestrate a Round-Robin Multi-Agent Team

LAB 7

LO3 — compare manual orchestration with a RoundRobinGroupChat team.

Hand-roll a two-model writer/critic conversation to feel the orchestration cost, then run the same workflow as a RoundRobinGroupChat with proper termination conditions and a readable TaskResult.

YOU'LL BUILD

A manual two-model conversation script, and a writer/reviewer team that stops on FINALISED.

TOOLCHAIN

RoundRobinGroupChat, TextMentionTermination, MaxMessageTermination, TaskResult

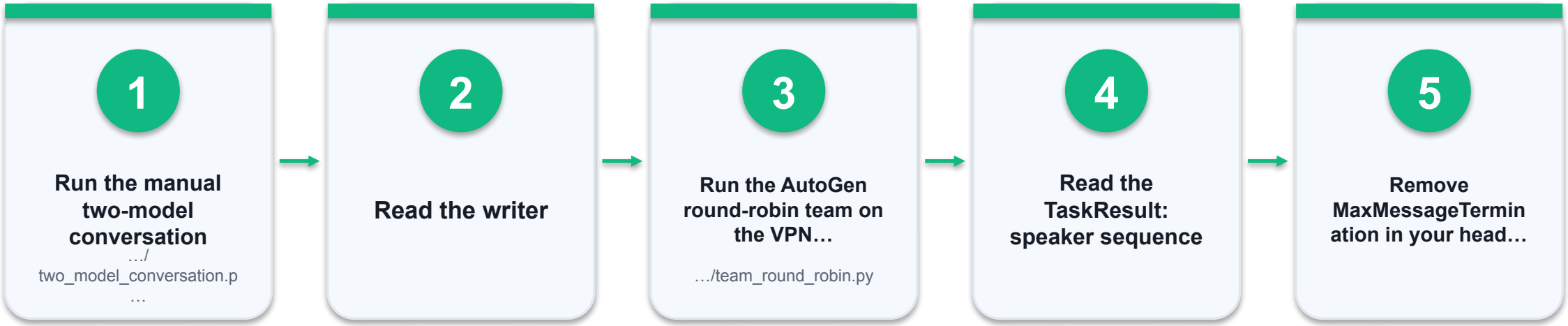
DONE WHEN

The team alternates writer → reviewer and stops on FINALISED (not the message cap), and you can read the stop reason straight from the TaskResult.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · PROCESS MAP

How Lab 7 Runs



THE POINT

LO3 — compare manual orchestration with a RoundRobinGroupChat team.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · PROCEDURE

Orchestrate a Round-Robin Multi-Agent Team — Part 1/2

1. **Run the manual two-model conversation and count the plumbing it needs**

```
$ uv run python lab-07-round-robin-team/two_model_conversation.py
```

2. Read the writer and reviewer system messages — the reviewer holds the checklist

3. **Run the AutoGen round-robin team on the VPN announcement task**

```
$ uv run python lab-07-round-robin-team/team_round_robin.py
```

4. Read the TaskResult: speaker sequence, message count and stop_reason

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · PROCEDURE

Orchestrate a Round-Robin Multi-Agent Team — Part 2/2

5 Remove MaxMessageTermination in your head: what could an unbounded team cost?

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · VERIFY

Orchestrate a Round-Robin Multi-Agent Team

✅ Expected result

The team alternates writer → reviewer and stops on FINALISED (not the message cap), and you can read the stop reason straight from the TaskResult.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the OPENAI_API_KEY into labs/.env — no quotes, no spaces.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Route Specialists with SelectorGroupChat (Capstone)

LAB 8

LO3 — build the OpsDesk capstone and assess its feasibility with checks.

Assemble the capstone: a planner plus ticket and policy specialists routed dynamically by SelectorGroupChat, combining tools and RAG — then run the no-LLM readiness checks that decide whether the workflow is fit to ship.

YOU'LL BUILD

The complete OpsDesk multi-agent workflow, and a readiness report covering data, index health, retrieval accuracy and secrets hygiene.

TOOLCHAIN

SelectorGroupChat, agent descriptions, FunctionTool + RAG, readiness checks, AutoGen Studio (optional)

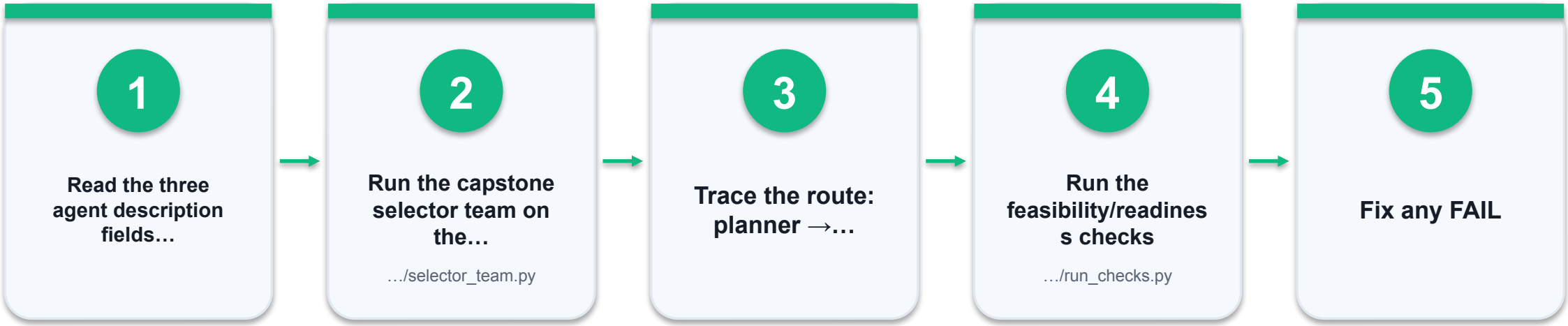
DONE WHEN

The team run ends on TERMINATE with a customer-ready reply citing POL-001, and run_checks.py prints 0 failed — your evidence-based feasibility verdict.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · PROCESS MAP

How Lab 8 Runs



THE POINT

LO3 — build the OpsDesk capstone and assess its feasibility with checks.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · PROCEDURE

Route Specialists with SelectorGroupChat (Capstone) — Part 1/2

1. Read the three agent description fields — the selector routes by reading them

2. **Run the capstone selector team on the TT-1002 VPN request**

```
$ uv run python lab-08-selector-workflow-capstone/selector_team.py
```

3. Trace the route: planner → ticket_specialist → policy_specialist → final reply

4. **Run the feasibility/readiness checks (no LLM needed)**

```
$ uv run python lab-08-selector-workflow-capstone/run_checks.py
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · PROCEDURE

Route Specialists with SelectorGroupChat (Capstone) — Part 2/2

5 Fix any FAIL (e.g. rebuild the Lab 4 index) and re-run until clean

6 **Optional: rebuild the team visually in AutoGen Studio**

```
$ uv run --with autogenstudio autogenstudio ui
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · VERIFY

Route Specialists with SelectorGroupChat (Capstone)

✅ Expected result

The team run ends on TERMINATE with a customer-ready reply citing POL-001, and run_checks.py prints 0 failed — your evidence-based feasibility verdict.

IF IT DOESN'T WORK

Nothing happens

Check the .env file is in the labs folder and the key has no quotes or spaces.

Auth or 401 error

Re-copy the OPENAI_API_KEY into labs/.env — no quotes, no spaces.

ModuleNotFoundError

Run uv sync again, and prefix commands with uv run so the venv is used.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Implementing a Multi-Agent AI Workflow

- You can now: expose the ReAct loop, then gate an agent behind human approval.
- You can now: compare manual orchestration with a RoundRobinGroupChat team.
- You can now: build the OpsDesk capstone and assess its feasibility with checks.

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Course Summary & Next Steps

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING OUTCOMES

What You Achieved

1

LO1 · Evaluated LLM AI models

Ran models directly and as agents, compared cloud vs local on quality, latency, cost and privacy, and identified strengths and limitations with evidence.

2

LO2 · Analyzed RAG algorithms

Built an embedding index, tuned chunking and top-k, grounded an agent with citations and refusal, and measured hit@k and latency to improve efficiency.

3

LO3 · Assessed multi-agent feasibility

Built ReAct, human-in-the-loop, round-robin and selector workflows, bounded them with termination conditions, and judged feasibility with readiness checks.

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

Continue Your Learning

1

AutoGen documentation

https://microsoft.github.io/autogen/ — the reference for AgentChat, Extensions and Core.

2

AI Agents for Beginners

Microsoft's free 10-lesson course on agent design patterns — github.com/microsoft/ai-agents-for-beginners.

3

Rebuild the labs

Redo each lab from a blank file until the agent patterns are automatic.

4

Extend the capstone

Add a new specialist, a guardrail and a Studio prototype to your OpsDesk workflow.

5

Practice exams

https://exams.tertiaryinfotech.com/practice-exams — free practice exams from Tertiary Infotech.

6

Share your work

Publish your agents to GitHub and gather feedback from real users.

© 2026 Tertiary Infotech Academy Pte Ltd

CONTINUE WITH TERTIARY INFOTECH

Recommended Courses

1. WSQ - Building Agentic AI Workflows to Automate Business Processes
2. WSQ - Agentic AI Automation with n8n
3. WSQ - Create Copilot Chatbots and Agents with Microsoft Copilot Studio
4. WSQ - Develop Multi AI Agent Applications with Gemini Agent ADK
5. WSQ - LLM Application Development with LangChain

© 2026 Tertiary Infotech Academy Pte Ltd

WE'RE HERE TO HELP

Support

- If you have any enquiries during or after the class, you can contact us below.
- Email: enquiry@tertiaryinfotech.com
- Tel: +65 6100 0613
- Website: www.tertiarycourses.com.sg
- LMS / TMS: https://lms-tms.tertiaryinfotech.com

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT

Final Assessment

1

Written Assessment (SAQ)

1 hour · open book · short-answer knowledge questions.

2

Practical Performance (PP)

1 hour · open book · hands-on AutoGen build tasks.

3

Digital attendance

Take the Assessment digital attendance before you start.

4

Submit on the LMS

Upload your completed answers at https://lms-tms.tertiaryinfotech.com

© 2026 Tertiary Infotech Academy Pte Ltd

ON ASSESSMENT DAY

Assessment Flow

1

TRAQOM survey

Scan the QR code on the…

2

Digital attendance

Scan the SSG assessment QR

3

Sit WA then PP

Open book · 1 hour each

4

Submit on the LMS

Upload your completed…

5

Sign the record

Sign the Assessment…

REMEMBER

All five steps are mandatory for WSQ funding — missing the digital attendance or the TRAQOM survey can invalidate your claim.

© 2026 Tertiary Infotech Academy Pte Ltd

Digital Attendance & TRAQOM (Mandatory)

1

Three times a day

Take the AM, PM and Assessment digital attendance — mandatory for every WSQ-funded course.

2

Trainer shows the QR

The trainer or administrator displays the digital attendance QR code from the SSG portal.

3

Scan and submit

Scan the QR code with your mobile phone camera and submit your attendance.

4

Cert & TRAQOM survey

Complete the TRAQOM survey on the LMS to receive your certificate.

© 2026 Tertiary Infotech Academy Pte Ltd

HAPPY BUILDING

Thank You!

You can now design, build and evaluate multi-agent AI applications with AutoGen.

© 2026 Tertiary Infotech Academy Pte Ltd